

Badanie efektywności algorytmów grafowych w zależności od rozmiaru grafu i sposobu reprezentacji

Krystian Mikołajczak 284452

19 czerwca 2026

1 Wstęp

1.1 Opis zadania

Celem było zaimplementowanie wybranych algorytmów rozwiązujących problemy grafowe. Rozwiązywane problemy to minimalne drzewo rozpinające i znalezienie najkrótszej ścieżki w grafie. Wyniki efektywności rozwiązywania zadanych problemów za pomocą wybranych algorytmów następnie zostały poddane analizie oraz porównane między sobą. Badana będzie efektywność algorytmów w zależności od ilości wierzchołków w grafie i od gęstości krawędzi w grafie.

1.2 Algorytmy poddane analizie

Analizie poddano dwa algorytmy dla każdego z dwóch problemów. Każdy z algorytmów został zbadany dla reprezentacji grafu w formie macierzy sąsiedztwa oraz w formie listy następników/poprzedników.

1.2.1 Minimalne drzewo rozpinające

Jest to drzewo zawierające wszystkie wierzchołki danego grafu o najmniejszej możliwej sumie wag krawędzi.

- Algorytm Prima

Algorytm Prima buduje strukturę krok po kroku, rozpoczynając od wybranego wierzchołka i zachłannie dołączając najbliższe krawędzie łączące odwiedzoną część grafu z nieodwiedzoną. Implementacja wykorzystująca macierz sąsiedztwa wymaga przeglądania całego wiersza (wszystkich V wierzchołków) przy każdym zdjęciu elementu z kolejki priorytetowej, co daje pesymistyczną złożoność rzędu $O(V^2 \log V)$. Zastosowanie listy następników pozwala z kolei na iterowanie wyłącznie po rzeczywistych krawędziach, co obniża czas do $O(E \log V)$. Zakłada się więc, że reprezentacja listowa będzie znacznie szybsza dla grafów rzadkich, podczas gdy macierzowa może być konkurencyjna w grafach gęstych z uwagi na mniejszy narzut pamięciowy i ciągły układ w pamięci. [1]

- Algorytm Kruskala

Algorytm Kruskala początkowo wyodrębnia wszystkie krawędzie grafu, sortuje je według wag w kolejce priorytetowej, a następnie łączy wierzchołki, korzystając ze struktury zbiorów rozłącznych (find-union) w celu unikania cykli. Sposób reprezentacji grafu determinuje narzut na początkowe pozyskanie krawędzi. Odczyt z macierzy sąsiedztwa zmusza algorytm

do sprawdzenia każdej możliwej pary wierzchołków ($O(V^2)$), nawet w przypadku braku krawędzi. Zastosowanie listy następników skraca ten proces jedynie do przeglądu istniejących połączeń $O(V + E)$. Oczekuje się zatem przewagi listy na etapie inicjalizacji dla grafów rzadkich, jednak w ostatecznym rozrachunku dla grafów gęstych o czasie wykonania w obu wersjach zadecyduje sam koszt sortowania krawędzi w kolejce ($O(E \log E)$). [2]

1.2.2 Znalezienie najkrótszej ścieżki

Znalezienie najkrótszego pod względem sumy wag połączenia między dwoma wierzchołkami w grafie.

- Algorytm Dijkstry
Algorytm Dijkstry przeszukuje graf za pomocą strategii zachłannej, wyciągając z kolejki priorytetowej wierzchołki o najmniejszym koszcie dotarcia i relaksując ich sąsiadów. Podobnie jak w algorytmie Prima, reprezentacja ma kluczowy wpływ na etap weryfikacji sąsiedztwa. Użycie macierzy sąsiedztwa oznacza konieczność iterowania po wszystkich V wierzchołkach dla każdego analizowanego węzła, co owocuje złożonością czasową w okolicach $O(V^2 \log V)$. Wersja na liście następników filtruje iteracje wyłącznie do fizycznych sąsiadów, osiągając optymalny czas $O(E \log V)$. Przewiduje się, że reprezentacja listowa zdecydowanie zdominuje macierzową w grafach rzadkich, natomiast w sieciach zbliżonych do grafów pełnych różnice w czasie wykonywania ulegną silnemu zatarciu. [3]
- Algorytm Forda-Bellmana
Algorytm Forda-Bellmana, bazujący na programowaniu dynamicznym, opiera się na wykonaniu $V - 1$ iteracji, w których próbuje zrelaksować wszystkie połączenia w grafie. Wybór reprezentacji determinuje tu drastyczne różnice w wydajności. W implementacji z macierzą sąsiedztwa mechanizm wymusza sprawdzanie w każdej z iteracji pełnej macierzy $V \times V$, co generuje fatalną w skutkach złożoność sześcienną $O(V^3)$. Zmiana struktury na listę następników sprawia, że w każdej głównej iteracji algorytm przebywa wyłącznie po istniejących krawędziach (E), redukując czas do poziomu $O(V \cdot E)$. Oczekuje się, że w przypadku tej metody reprezentacja listowa zmiażdży macierzową pod kątem wydajności na niemal każdym badanym zbiorze danych. [4]

1.3 Charakterystyka danych badawczych

Dane wykorzystane w badaniu zostały wygenerowane przy użyciu generatora liczb pseudolosowych. Podstawowym badanym typem danych dla wag krawędzi jest format liczb całkowitych (int, 4 bajty). Badanie obu algorytmów rozwiązujących ten sam problem zostało przeprowadzone na tych samych instancjach reprezentacji. Oba typy reprezentacji opisują ten sam badany graf.

W badaniu uwzględniono trzy reprezentatywne poziomy gęstości grafu:

- 25%
- 50%
- 99%

Badania wykonano dla siedmiu reprezentatywnych rozmiarów grafu: 50, 100, 200, 400, 600, 800, 1000 wierzchołków. Dla każdej kombinacji parametrów badanie wykonano na 50 wygenerowanych instancjach problemu.

2 Warunki badawcze

2.1 Środowisko badawcze

Badanie zostało przeprowadzone za pomocą autorskiego oprogramowania napisanego w języku C++, skompilowanego za pomocą kompilatora Apple Clang w wersji 17.0.0 ,zgodnie ze standardem C++20, stosując standardową konfigurację. Specyfikacja platformy badawczej:

- **Procesor:** Apple M4, 10 rdzeni (4 rdzenie wydajnościowe, 6 rdzeni energooszczędnych)
- **Pamięć operacyjna:** 16 GB zunifikowanej pamięci RAM

Literatura

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L Rivest, Clifford Stein: Wprowadzenie do algorytmów, PWN, 2024, s. 573-574.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L Rivest, Clifford Stein: Wprowadzenie do algorytmów, PWN, 2024, rozdział 21.
- [3] Wikipedia Algorytm Dijkstry - dostęp 19.06.2026 (pl.wikipedia.org/wiki/Algorytm_Dijkstry)
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L Rivest, Clifford Stein: Wprowadzenie do algorytmów, PWN, 2024, s.664-665.